

Topic 4: The Transformer Architecture

4.1 Why Transformers Changed Everything

- Before 2017, sequence models (RNNs, LSTMs) processed text one token at a time -- slow and hard to parallelise
- The Transformer (Vaswani et al., 2017) processes all tokens in a sequence simultaneously using self-attention
- This allowed training on vastly larger datasets using modern GPU hardware at scale
- Transformers became the foundation for GPT, BERT, T5, Claude, Gemini, and almost every modern LLM
- Original paper: <https://arxiv.org/abs/1706.03762>

4.2 Self-Attention -- The Core Mechanism

- Self-attention allows each token to 'look at' all other tokens in the sequence when computing its representation
- For each token, the model computes Query, Key, and Value vectors from the token's embedding
- Attention scores: dot product of Query with all Keys, scaled and softmaxed to produce weights that sum to 1
- Output: weighted sum of Values, where higher-scoring tokens contribute more
- This lets the model capture long-range dependencies -- the word 'it' can attend to 'the bank' 20 tokens earlier

Simplified attention for one token:

```
scores = softmax( Query . Key^T / sqrt(d_k) )  
output = scores . Values
```

The result: each token's output is a mix of all other tokens, weighted by how relevant they are to each other.

4.3 Key Components of a Transformer

- Token embeddings: convert each token (word/sub-word) into a dense numeric vector
- Positional encodings: inject information about each token's position (since attention is order-agnostic)
- Multi-head attention: run self-attention in parallel with different learned projections to capture different relationships

- Feed-forward layers: applied after attention in each block; add capacity for learning complex transformations
- Layer normalisation: stabilises training by normalising activations at each layer
- Residual connections: skip connections that help gradients flow during training of deep stacks

4.4 Encoder, Decoder, and Encoder-Decoder

- Encoder-only (e.g., BERT): processes input bidirectionally; used for classification, search, embeddings
 - Decoder-only (e.g., GPT, Claude, Llama): generates text autoregressively -- predicts the next token given all previous tokens
 - Encoder-decoder (e.g., T5, BART): encoder reads input, decoder generates output -- used for translation and summarisation
 - Modern LLMs are almost all decoder-only: they are trained to predict the next token on massive text corpora
 - Context window (e.g., 128k tokens in GPT-4, 200k in Claude) limits how much text can be processed at once
-